허공 손글씨 계산기

손 궤적 인식 · 조기예측 모델 개발 및 시각화

작성자 : 김주영



1. 개요 및 현황

사업과제	AI 기반 허공 손글씨 계산기 – 손 궤적 인식·조기예측 모델 개발 및 시각화

추진배경 및 목적

- 키보드·마우스 없이 웹캠 앞에서 검지로 숫자(0~9)와 연산자(+ − × ÷ =)를 허공에 쓰면 인식해 계산해 주는 비접촉 입력 시스템
- MediaPipe Hands(사전학습 딥러닝)로 손 관절 21개를 추적하고, 손 모양 규칙으로 쓰기/이동/완료/지우기를 구분
- 궤적을 두 가지 형태(28×28 이미지, 64점 시퀀스)로 바꿔 CNN + BiGRU 하이브리드 딥러닝으로 15개 기호 분류
- 글자를 다 쓰기 전 앞부분(20~70%)만으로 기호와 나머지 궤적을 미리 예측하는 '조기 예측' 모델을 4종 비교·분석
- 혼자 수집 가능한 주제로 데이터 수집 → 학습 → 평가 → 웹앱(Streamlit) 배포까지 전 과정을 직접 수행

2. 과제 범위

과제구분		내용
AI	하이브리드 손글씨 인식 · 조기예측 모델 구현	웹캠 허공 궤적 수집 (세션 단위, 15개 기호 × 4바퀴)
		궤적 전처리 : 정규화 · 28×28 렌더 · 64점 리샘플 · 증강
		MNIST 사전학습 CNN + BiGRU 하이브리드 학습 · 앙상블
		세션 단위 교차검증 (macro F1, 숫자/연산자 정확도)
		조기 예측 모델 비교 (GRU · LSTM · TCN · Transformer)
		궤적 예측 정확도 분석 (ADE · FDE · Chamfer)
		확인용(holdout) 세션 최종 테스트
시각화	실시간 웹 계산기 및 결과 시각화	브라우저 카메라 연계 실시간 인식 (streamlit-webrtc)
		앱 화면 : 예측 확률 · 조기 예측 궤적(점선) 표시
		학습 결과 시각화 : 모델 비교 · 혼동행렬 · HTML 리포트
		얼굴 자동 모자이크 (개인정보 보호, 영상 미저장)
		Streamlit Community Cloud 배포 및 통합 테스트

3. 과제 추진 방법 – 주제 선정 기준

데이터 수집 용이성

- 혼자서 웹캠 하나로 수집 가능 (세션 1회 = 60개, 약 10분)
- 영상이 아닌 손끝 좌표만 저장 → 용량이 작고 개인정보 부담이 적음
- MNIST 공개 데이터로 숫자 형태를 사전학습해 적은 데이터를 보완

딥러닝 모델 적합성

- 같은 궤적을 '모양(이미지)'과 '순서(시퀀스)' 두 관점으로 볼 수 있어 하이브리드 구조에 적합
- 쓰는 중인 앞부분으로 미래 궤적을 맞히는 시계열 '예측' 문제로 확장 가능
- 15개 클래스, 짧은 시퀀스로 CPU에서도 학습 가능한 규모

활용성 및 확장성

- 손이 더럽거나 화면을 만질 수 없는 환경(주방·실험실·의료)의 비접촉 입력
- 브라우저만 있으면 실행되는 웹앱으로 배포 → 누구나 바로 체험
- 기호 클래스를 늘리면 한글·서명·제스처 명령 입력으로 확장 가능

3. 과제 추진 방법 – AI 예측 분석모델 적용 대상

적용 기능	수집데이터	예측모델 입력 (독립변수)	AI 예측 분석 대상
허공 손글씨 계산기	• 웹캠 30fps 손 관절 21개 좌표 (MediaPipe) • 검지 끝 궤적 (t, x, y) + 획 구분 • 학습 4세션 × 60개 = 240개 • 확인용 1세션 60개 (학습 미사용) • 합성 연산자 120개/기호 (학습 fold만)	• 이미지 : 28×28 흑백 렌더 (MNIST 형식) • 시퀀스 : 64점 × 6특징 (x, y, dx, dy, 획시작, 진행률) • 조기 예측 : 앞부분 32점 절대좌표 • 손 모양 : 손가락 PIP 각도 (>140°)	• 15개 기호 분류 (숫자 10 + 연산자 5) • 쓴 비율별 조기 인식 정확도 • 나머지 궤적 48점 좌표 예측 • 앙상블 확신도 기반 자동 입력 • 수식 계산 (분수 · 연산자 우선순위)

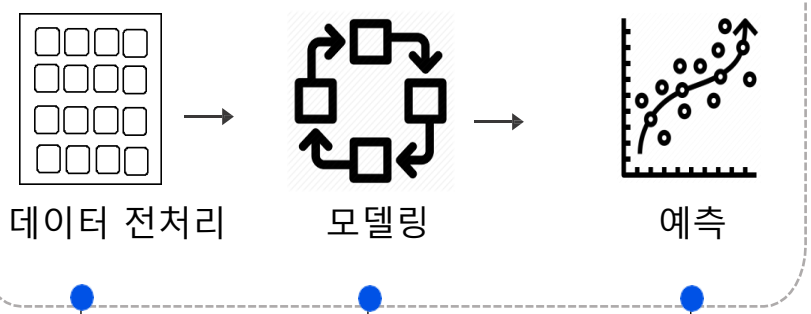
3. 과제 추진 방법 – AI 예측 분석모델 구축 프로세스

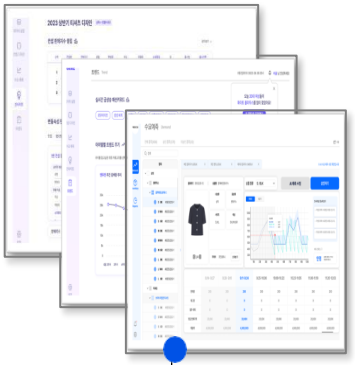
DATA IMPORTING

AUTO PREDICTION

USER INSIGHT







- 웹캠 손 궤적 수집
- collect_data.py
- 세션 단위 저장 (.npz)
- 확인용 세션 별도 보관

- 궤적 정규화
- 28×28 이미지 렌더
- 64점 리샘플 · 증강
- 세션 단위 fold 분할

- MNIST 사전학습 CNN
- BiGRU + Attention
- 가중 앙상블 (검증셋 최적)
- 조기예측 4종 비교

- 15개 기호 분류
- 작성 비율별 조기 인식
- 나머지 궤적 예측
- macro F1 · ADE 평가

- Streamlit 웹 계산기
- 실시간 확률·궤적 표시
- HTML 결과 리포트
- 클라우드 배포

4. 주요 결과물 – 라이브러리 구조 및 사용 스택

프로젝트 구조 · 기술 스택

- aircalc/ 패키지에 핵심 기능(손 추적·전처리·모델·계산기)을 모으고, scripts/는 ①수집 → ②학습 → ③조기예측 → ④확인 순서로 실행
- 같은 aircalc 코드를 수집 프로그램 · 학습 · 웹앱이 함께 사용 → 학습 때와 실제 앱의 전처리가 항상 동일

```
AirWrite_Net_VisionCalc/
├ app.py                  # Streamlit 웹앱 (브라우저 카메라)
├ requirements.txt        # 배포용 버전 고정
├ packages.txt · .python-version · .streamlit/
├ aircalc/                # 핵심 라이브러리 패키지
│  ├ config.py           # 경로·하이퍼파라미터·제스처 설정
│  ├ hand_tracking.py    # MediaPipe Hands + 손 모양 규칙
│  ├ strokes.py          # AirWriter, 28×28 렌더, 64점 리샘플, 증강
│  ├ drawing.py          # 얼굴 모자이크, 궤적·점선 그리기
│  ├ models.py           # StrokeCNN · StrokeGRU · 앙상블
│  ├ recognizer.py       # 실시간 인식기 (앱/수집 공용)
│  ├ calculator.py       # 분수 계산 + 연산자 우선순위
│  ├ early.py · early_models.py   # 조기 예측 EarlyNet
│  └ report.py · early_report.py  # HTML 결과 리포트
├ scripts/
│  ├ collect_data.py      # ① 데이터 수집 (세션 단위)
│  ├ train.py             # ② CNN+GRU 하이브리드 학습
│  ├ train_early.py       # ③ 조기 예측 모델 비교 학습
│  └ evaluate_holdout.py · evaluate_early_holdout.py  # ④ 확인용
├ models/   # cnn.pt, gru.pt, ensemble.json, early_*.pt
├ outputs/  # 그래프·혼동행렬·HTML 리포트
└ data/     # 수집 궤적 (.gitignore, 영상 저장 X)
```

언어 · 개발환경
- Python 3.12
- VS Code
- Git Bash
- GitHub

컴퓨터 비전
- OpenCV
- MediaPipe Hands 0.10.21
- Haar 얼굴 검출

딥러닝
- PyTorch 2.6 (CPU)
- torchvision (MNIST)
- scikit-learn

데이터 · 시각화
- NumPy
- pandas
- Matplotlib
- HTML 리포트

웹 · 배포
- Streamlit 1.60
- streamlit-webrtc
- Streamlit Cloud

AI 협업 도구
- Claude (설계·코드 리뷰·디버깅)

4. 주요 결과물 – 사용 모델 및 AI 구조

하이브리드 AI 파이프라인

- 사전학습 딥러닝(MediaPipe) + 규칙(손 모양) + 직접 학습한 딥러닝 2종(CNN·BiGRU)을 결합한 하이브리드 구조
- CNN은 완성된 '모양', BiGRU는 '쓰는 순서·방향'을 보고 판단 → 서로 약점을 보완하도록 검증셋에서 비중 결정



4. 주요 결과물 – 데이터 수집

손 제스처 인식 규칙 (입력 방식)

MediaPipe가 찾은 손 관절로 손가락 펴짐을 판별 → 학습 없이 규칙만으로 4가지 손 모양을 구분
쓰기 시작은 3프레임, 멈춤은 2프레임 연속일 때 확정(비대칭 디바운스) → 획이 이어지거나 끊기는 문제 해결

```python
def classify_gesture(states):           # 손가락 펴짐(PIP 각도 > 140°) → 손 모양
    idx, mid, ring, pinky = states["index"], states["middle"], states["ring"], states["pi
    if idx and not (mid or ring or pinky): return "draw"     # ☝ 쓰기
    if idx and mid and not (ring or pinky): return "hover"   # ✌ 이동
    if idx and mid and ring and pinky:      return "confirm" # 🖐 완료
    if not (idx or mid or ring or pinky):   return "fist"    # ✊ 지우기
```

```python
# AirWriter._stable_gesture : 쓰기 '시작'은 3프레임, '멈춤'은 2프레임 (비대칭 디바운스)
if self.gesture == "draw":
    self._off_count = self._off_count + 1 if raw != "draw" else 0
    if self._off_count >= GESTURE_RELEASE_FRAMES:      # 2프레임만 벗어나도 획을 끊음
        self.gesture, self._off_count = raw, 0
    return self.gesture
if self._candidate_count >= GESTURE_STABLE_FRAMES:     # 3프레임 유지돼야 시작
    self.gesture = self._candidate
```

☝ 검지 = 쓰기　✌ 검지+중지 = 이동(펜 들기)　🖐 손바닥 = 글자 완료　✊ 주먹 0.6초 = 지우기

4. 주요 결과물 – 데이터 수집

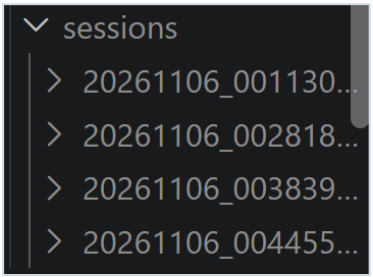
데이터 수집 (collect_data.py)

- 학습 세션 4회 × 60개 = 240개 (15개 기호 × 4바퀴, 기호 순서는 바퀴마다 무작위) + 확인용(holdout) 세션 60개
- 세션마다 조건을 바꿔 다양성 확보 (예: '작게 빠르게', '서서') · 영상은 저장하지 않고 궤적 좌표만 .npz로 저장

```
$ python scripts/collect_data.py --note "확인용" --holdout
 manager requires a model with a single signature inference. Disabling support fo
r feedback tensors.
============================================================
세션 20261106_005914_me  [확인용] | 총 60개 (15개 기호 x 4바퀴)
검지만 펴서 쓰기 / V손으로 이동 / 손바닥 펴서 저장 / 주먹으로 지우기
키: space=저장  c=지우기  r=방금 저장 삭제  s=건너뛰기  q=종료
============================================================
[1/60] 쓸 기호: 1
W0000 00:00:1793894368.643915   41040 landmark_projection_calculator.cc:186] Usin
g NORM_RECT without IMAGE_DIMENSIONS is only supported for the square ROI. Provid
e IMAGE_DIMENSIONS or use PROJECTION_MATRIX.
   저장: 1 (획 1개, 점 85개)
[2/60] 쓸 기호: x   (↘ 대각선 → (V 손으로 이동) → ↙ 대각선)
   저장: x (획 2개, 점 29개)
[3/60] 쓸 기호: +   (가로 → (V 손으로 이동) → 세로)
   저장: + (획 2개, 점 18개)
[4/60] 쓸 기호: 2
   저장: 2 (획 1개, 점 44개)
[5/60] 쓸 기호: 4
   저장: 4 (획 2개, 점 42개)
[6/60] 쓸 기호: 5
   저장: 5 (획 1개, 점 61개)
[7/60] 쓸 기호: 7
   저장: 7 (획 2개, 점 61개)
[8/60] 쓸 기호: =   (위 가로 → (V 손으로 이동) → 아래 가로)
```

확인용 세션 수집 터미널 출력 (기호별 획 수 · 점 수 기록)





저장 구조

data/sessions/ : 학습용 4세션

data/holdout/ : 확인용 (학습에 절대 사용 X)

기호별 폴더 + session_info.json

수집 데이터 폴더 구조

4. 주요 결과물 – 데이터 분석

데이터 구성 및 평가 방식

- 전체 240개 = 4세션 × 15클래스 × 4개 (불균형 없음)
- 피험자 1명 → '세션 단위' 분할 (StratifiedGroupKFold)
- 시험 세션은 해당 fold 모델이 처음 보는 날의 글씨
- 연산자는 MNIST에 없어 합성 120개/기호로 보충 (학습 fold만)
- MNIST로 CNN 사전학습 3 epoch → 시험 정확도 98.9%
- 평가지표 : macro F1, 숫자/연산자별 · 쓴 비율별 정확도

```
$ python scripts/train.py
전체 240개, 세션 4개
class              0  1  2  3  4  5  6  7  8  9  +  -  x  /  =
session
20261106_001130_me  4  4  4  4  4  4  4  4  4  4  4  4  4  4  4
20261106_002818_me  4  4  4  4  4  4  4  4  4  4  4  4  4  4  4
20261106_003839_me  4  4  4  4  4  4  4  4  4  4  4  4  4  4  4
20261106_004455_me  4  4  4  4  4  4  4  4  4  4  4  4  4  4  4
[안내] 피험자가 적어 세션 단위로 나눠 평가합니다 (혼자 수집하면 정상).
MNIST로 CNN 사전학습 중 (3 epoch, 처음 한 번만)...
  epoch 1: MNIST 시험 정확도 0.9862
  epoch 2: MNIST 시험 정확도 0.9898
  epoch 3: MNIST 시험 정확도 0.9892
```

train.py 실행 결과 – 세션 × 클래스 표본 수 (모든 칸 4개로 균형)

```
 …
[안내] 피험자가 적어 세션 단위로 나눠 평가합니다 (혼자 수집하면
 정상).
MNIST로 CNN 사전학습 중 (3 epoch, 처음 한 번만)...
```

세션 단위 평가 안내 및 MNIST 사전학습 로그

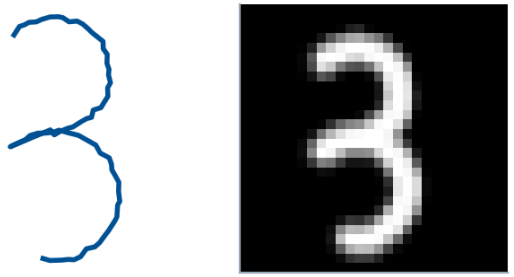
4. 주요 결과물 – 탐색적 데이터 분석(EDA)

데이터 증강 (Augmentation)

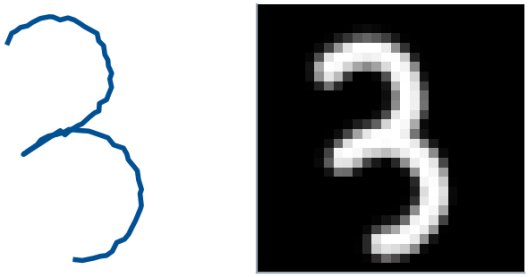
학습할 때마다 무작위로 변형해 적은 데이터로도 다양한 필체를 학습 (strokes.augment)

· 회전은 ±12°로 제한 → '+'와 '×'(45° 관계)가 뒤바뀌지 않도록

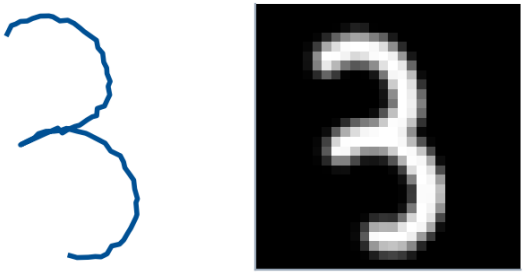
원본



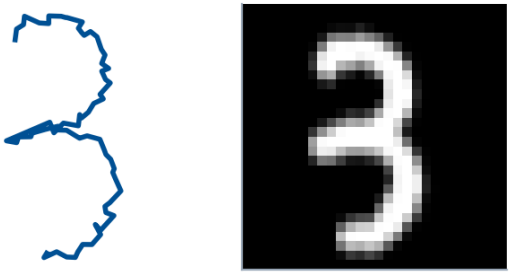
회전 ±12°



기울임(전단)



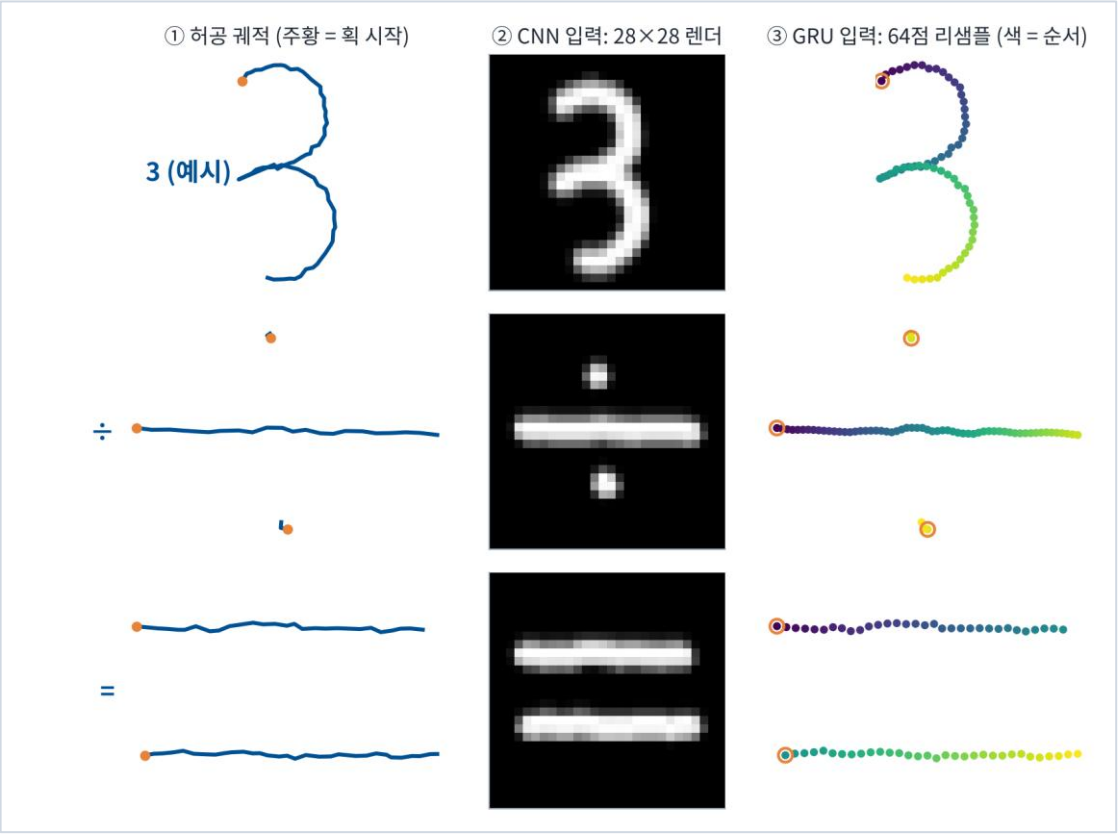
손떨림(지터)



왼쪽 = 변형된 궤적, 오른쪽 = CNN에 들어가는 28×28 이미지 (프로젝트 코드로 생성)

4. 주요 결과물 – 탐색적 데이터 분석(EDA)

데이터 전처리



① 정규화
· 글자 중심을 원점, 긴 변 절반을 1로
· 가로세로 비율 유지 ('–'은 납작하게)

② CNN 입력 : 28×28 이미지
· 4배 크게 그린 뒤 축소 (부드러운 선)
· MNIST처럼 무게중심을 가운데로
· 짧은 획은 점으로 (÷의 점)

③ GRU 입력 : 64점 시퀀스
· 획 길이 비율로 점 배분 후 균등 재추출
· → 천천히/빨리 써도 같은 입력
· 특징 6개 : x, y, dx, dy, 획시작, 진행률

④ 꼬리 제거
· 손 모양을 바꾸는 순간 찍힌 마지막 점 삭제

4. 주요 결과물 – 데이터 학습 및 모델정의

모델정의 및 컴파일

CNN + BiGRU 모델 정의

☑ CNN은 합성곱 블록 2개로 모양을, BiGRU는 점 순서를 양방향으로 학습

```
class StrokeCNN(nn.Module):          # 모양을 보는 CNN (28x28)
    def __init__(self, num_classes, dropout=0.3):
        def block(cin, cout):
            return nn.Sequential(
                nn.Conv2d(cin, cout, 3, padding=1), nn.BatchNorm2d(cout), nn.ReLU(),
                nn.Conv2d(cout, cout, 3, padding=1), nn.BatchNorm2d(cout), nn.ReLU()
                nn.MaxPool2d(2))
        self.features = nn.Sequential(block(1, 32), block(32, 64))   # 28 -> 14 -> 7
        self.embed = nn.Sequential(nn.Flatten(), nn.Dropout(dropout),
                                   nn.Linear(64 * 7 * 7, 128), nn.ReLU())
        self.head = nn.Linear(128, num_classes)          # 숫자 10 + 연산자 5
```

```
class StrokeGRU(nn.Module):          # 쓰는 순서를 보는 BiGRU
    def __init__(self, num_classes, input_size=6, hidden=64):
        self.gru  = nn.GRU(input_size, hidden, batch_first=True,
                           bidirectional=True)
        self.pool = AttentionPool(hidden * 2)    # 중요한 점에 가중치
        self.head = nn.Sequential(nn.LayerNorm(hidden * 2),
                                  nn.Linear(hidden * 2, num_class

    def forward(self, x):                    # x: (B, 64점, 6특징)
        h, _ = self.gru(x)
        return self.head(self.pool(h))
```

모델 컴파일 (손실함수 · 최적화 · 앙상블 비중)

☑ 라벨 스무딩 교차엔트로피 + AdamW, 검증 세트로 CNN 비중 선택

```
def choose_weight(p_cnn, p_gru, y):     # 검증 세트에서 최적 CNN 비중 탐색
    for w in [0.0, 0.25, 0.5, 0.75, 1.0]:
        f1 = f1_score(y, ensemble(p_cnn, p_gru, w).argmax(1), average="macro")
        ...
criterion = nn.CrossEntropyLoss(label_smoothing=0.05)
opt   = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=epochs)
```

4. 주요 결과물 – 데이터 학습 및 모델정의

모델학습 및 시각화

모델 학습 (train.py)

- 세션 단위 4-fold 학습, 검증 세션으로 조기종료 · 앙상블 비중 결정

```
folds = StratifiedGroupKFold(4, shuffle=True).split(X, y, groups=sessions)  # 세션 단위 분할
for k, (train_idx, test_idx) in enumerate(folds, 1):
    syn_strokes, syn_y = make_synthetic(120, rng)        # 합성 연산자 (학습 fold에만)
    train_strokes = [strokes[i] for i in fit_idx] + syn_strokes
    for kind, pre in [("cnn", mnist_weights), ("gru", None)]:
        model, best_epoch = fit(kind, train_strokes, train_y, val_X, val_y,
                                epochs=60, pretrained=pre)  # 조기종료 patience 12
        probs[kind] = predict_proba(model, precompute(kind, test_strokes))
    w = choose_weight(val_probs["cnn"], val_probs["gru"], y[val_idx])
    probs["ensemble"] = ensemble(probs["cnn"], probs["gru"], w)
```

```
=== Fold 1/4: 시험 = 20261106_002818_me
  cnn macroF1=0.966 | gru macroF1=0.898 | ensemble macroF1=0.983 | 앙상블 비중 CNN:GRU = 0.50:0.50
  조기 예측 (앙상블 정확도): 25% 0.250 | 50% 0.333 | 75% 0.800 | 100% 0.983

=== Fold 2/4: 시험 = 20261106_004455_me
  cnn macroF1=0.966 | gru macroF1=0.731 | ensemble macroF1=0.966 | 앙상블 비중 CNN:GRU = 0.75:0.25
  조기 예측 (앙상블 정확도): 25% 0.250 | 50% 0.500 | 75% 0.800 | 100% 0.967

=== Fold 3/4: 시험 = 20261106_001130_me
  cnn macroF1=0.966 | gru macroF1=0.929 | ensemble macroF1=0.966 | 앙상블 비중 CNN:GRU = 1.00:0.00
  조기 예측 (앙상블 정확도): 25% 0.217 | 50% 0.400 | 75% 0.817 | 100% 0.967

=== Fold 4/4: 시험 = 20261106_003839_me
  cnn macroF1=0.930 | gru macroF1=0.841 | ensemble macroF1=0.930 | 앙상블 비중 CNN:GRU = 1.00:0.00
  조기 예측 (앙상블 정확도): 25% 0.167 | 50% 0.517 | 75% 0.733 | 100% 0.933

최종 CNN 학습: 실제 240개 + 합성 600개, 11 epoch

최종 GRU 학습: 실제 240개 + 합성 600개, 37 epoch
```

실행 결과 – fold별 macro F1 · 앙상블 비중 · 조기 예측 정확도, 최종 학습 epoch

4. 주요 결과물 – 데이터 학습 및 모델정의

모델학습 및 시각화

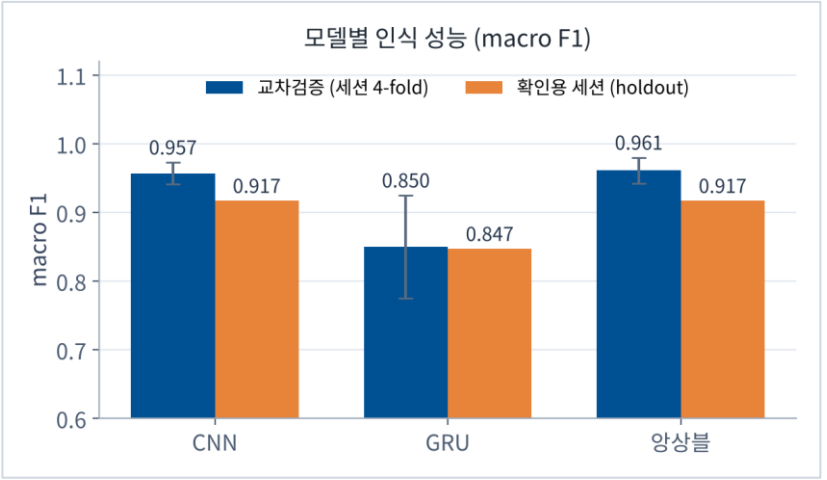
학습 결과 (교차검증 vs 확인용 세션)

- 앙상블 macro F1 0.961(교차검증) → 0.917(확인용), CNN:GRU = 0.88:0.12

```
======================================================================
교차검증 결과 (session 단위, 4 fold)
======================================================================
cnn          macro F1 0.957 ± 0.016 | 숫자 0.944 | 연산자 0.988
gru          macro F1 0.850 ± 0.075 | 숫자 0.825 | 연산자 0.925
ensemble     macro F1 0.961 ± 0.019 | 숫자 0.950 | 연산자 0.988
앙상블 비중 CNN:GRU = 0.88:0.12 | 글자당 CPU 11.0ms

조기 예측 정확도: 쓴 비율별 '전체 (숫자/연산자)'
                        25%                 50%                 75%                100%
cnn           0.212 (0.16/0.31)   0.421 (0.40/0.46)   0.792 (0.79/0.80)   0.958 (0.94/0.99)
gru           0.242 (0.19/0.34)   0.433 (0.44/0.42)   0.725 (0.70/0.78)   0.858 (0.82/0.93)
ensemble      0.221 (0.18/0.31)   0.438 (0.43/0.46)   0.788 (0.79/0.79)   0.962 (0.95/0.99)
앙상블 자동 입력 기준 (확신도 >= 90%, minus, 1 제외):
   25% 썼을 때: 2% 자동 입력, 그중 정답 100%
   50% 썼을 때: 5% 자동 입력, 그중 정답 70%
   75% 썼을 때: 23% 자동 입력, 그중 정답 99%
  100% 썼을 때: 42% 자동 입력, 그중 정답 99%
```

train.py 교차검증 결과 출력



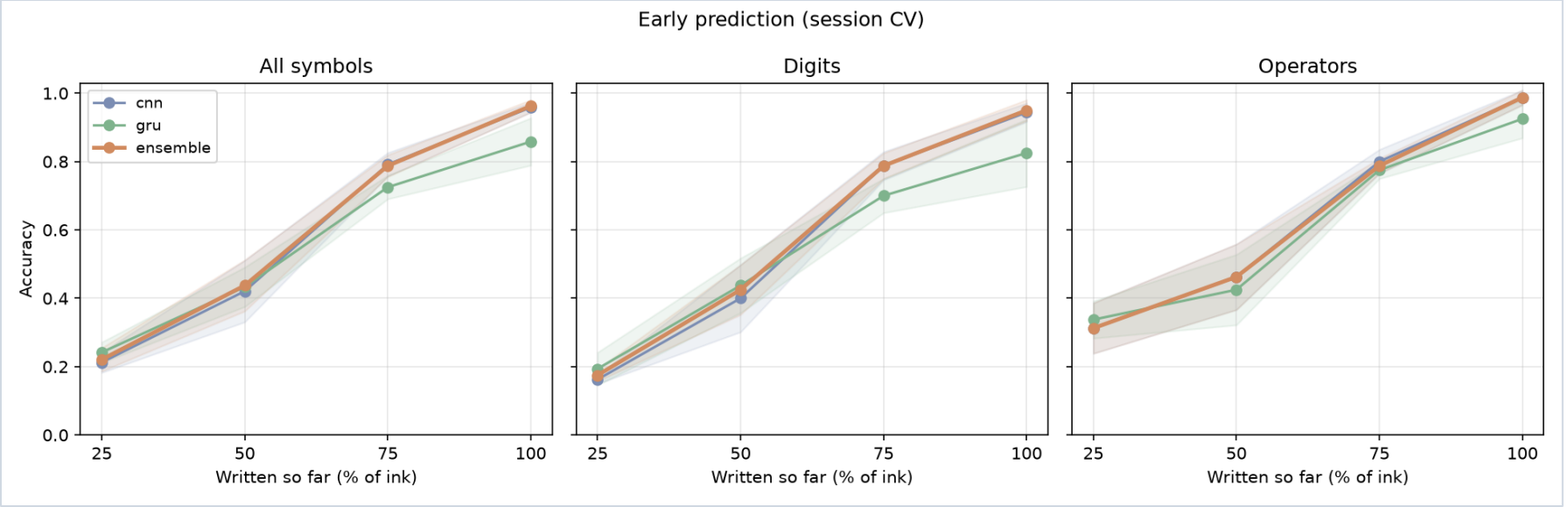
- 확인용 하락(−4.4%p)은 처음 보는 날 글씨라 정상 범위
- 연산자는 0.988로 높고, 숫자(0.950)에서 주로 오답
- 확인용 60개 기준 1개 = 약 1.7%p

4. 주요 결과물 – 데이터 학습 및 모델정의

모델 예측 – 조기 예측

작성 비율별 예측 정확도 (예측값 vs 실제 기호)

☑ 25 · 50 · 75 · 100%만 쓴 상태의 정답률 – 75%부터 0.79로 급상승



교차검증 앙상블 : 25% 0.22 → 50% 0.44 → 75% 0.79 → 100% 0.96 | 확인용 : 25% 0.23 → 50% 0.47 → 75% 0.72 → 100% 0.92

자동 입력 기준(확신도 ≥ 90%) : 75% 썼을 때 23%가 자동 입력되고 그중 99% 정답 → 다 쓰기 전에 입력되는 기능의 근거

4. 주요 결과물 – 데이터 학습 및 모델정의

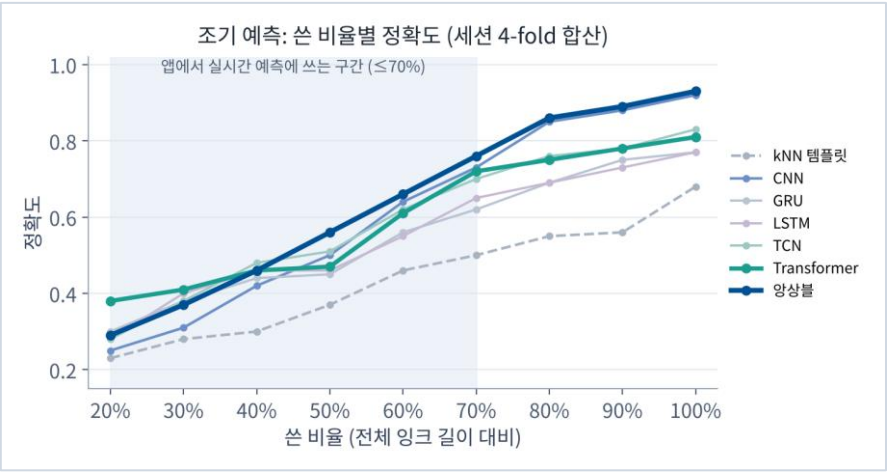
모델 예측 – 조기 예측 전용 모델

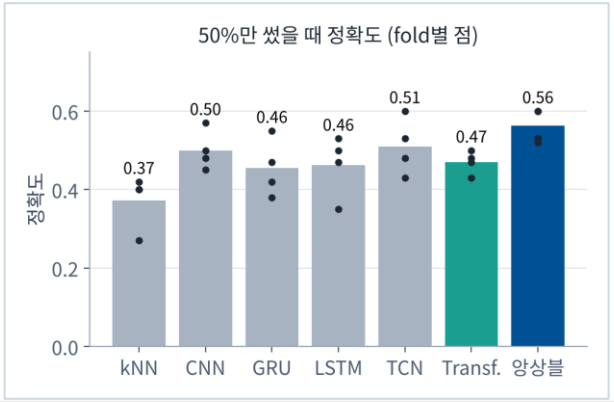
조기 예측 전용 모델 비교 (train_early.py)

- 앞부분으로 기호 + 나머지 궤적을 함께 예측 (EarlyNet, 인코더 4종 비교)

```python
class EarlyNet(nn.Module):      # 앞부분만 보고 "무슨 기호 + 나머지 궤적" 동시 예측
    def __init__(self, encoder, num_classes, length, out_points=48, hidden=96):
        self.encoder = {"gru": GRU, "lstm": LSTM, "tcn": TCN, "transformer": Transformer}[encoder]
        self.pool = AttentionPool(size)
        self.cls  = nn.Sequential(nn.LayerNorm(size), nn.Linear(size, num_classes))
        self.traj = nn.Sequential(nn.Linear(size + num_classes, 256), nn.ReLU(),
                                  nn.Linear(256, 256), nn.ReLU(), nn.Linear(256, out_points * 3))

    def forward(self, x):                       # x: 쓴 비율 20~100% 앞부분 (32점)
        h = self.pool(self.encoder(x))
        logits = self.cls(h)                                    # 분류 헤드
        traj = self.traj(torch.cat([h, logits.softmax(1).detach()], 1))
        return logits, traj.view(-1, 48, 3)      # (x, y, 획 시작) 48점
```





- 앱용 순서 모델 : Transformer (70% 이하 평균 0.508)
- 50% 지점은 효과가 엇갈림 (확인용 46.7% vs 45.0%)
- kNN 템플릿 기준선보다 모든 딥러닝 모델이 우수

4. 주요 결과물 – 프로토타이핑(화면)

결과 확인 화면

- 시험 샘플 확인 페이지 (outputs/test_samples.html)
  - ☑ 실제 궤적 · CNN 입력 이미지 · 상위 3개 확률 · CNN/GRU 의견 표시

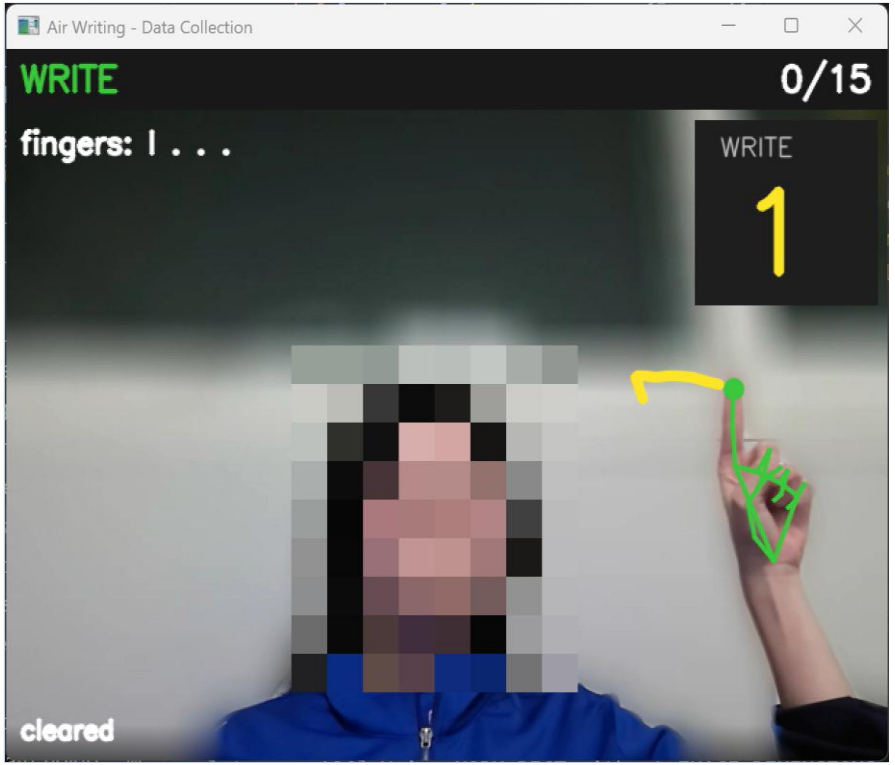


4. 주요 결과물 – 프로토타이핑(화면)

웹 계산기 (Streamlit)

실시간 허공 손글씨 계산기 (app.py)

☑ 쓰는 중 예측 확률 · 예상 궤적(점선) 표시, 손바닥을 펴면 입력 · 계산



화면 구성

· 카메라 카드 : 손 관절 · 궤적 · 얼굴 모자이크
· 계산기 디스플레이 : 수식과 결과 (분수 지원)
· 실시간 예측 카드 : 확신도 · 쓴 비율 막대
· 마지막 인식 : 상위 3개 + CNN/GRU 칩

기능

· ☝ 쓰기 · ✌ 이동 · 🤚 완료 · ✊ 지우기
· 조기 예측 궤적 점선 표시 (Transformer)
· 사이드바 : 모자이크/조기예측 토글, 모델 정보

디자인 · 배포

· 다크 테마 (Pretendard 폰트)
· Streamlit Community Cloud (Python 3.12)

5. 트러블 슈팅 ① 인식 · 코드 구조

획이 끊기지 않고 이어짐

- 문제 : ✌로 바꿔도 '쓰기'가 유지돼 다음 획과 선이 이어짐
- 해결 : 멈춤 2프레임 확정(비대칭 디바운스), 각도 150→140°, 3D 각도

목표 기호 · 획순 안내 사라짐

- 문제 : 코드 붙여넣기 중 수집 화면의 예시 기호 표시가 빠짐
- 해결 : draw_target() 분리, 합성 연산자로 획순 예시 (÷는 직접 그림)

폴더 분리 후 ImportError

- 문제 : aircalc/ · scripts/ 분리 후 import 실패, 설정값 누락
- 해결 : 상대 import + ROOT 경로 통일, config.py에 조기예측 설정 추가

5. 트러블 슈팅 ② 설치 · 배포 환경

pip 설치 오류 (cp949)

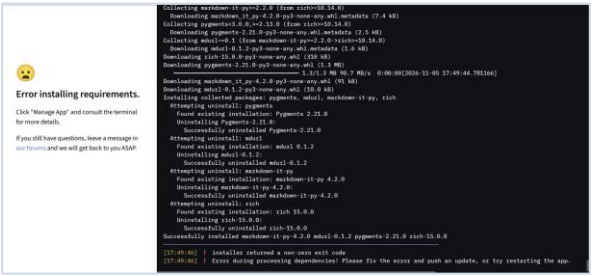
- 문제 : Windows가 requirements.txt 한글 주석을 cp949로 읽어 실패
- 해결 : 주석을 영어로 변경 (또는 PYTHONUTF8=1), 설치는 uv pip로 단축

MediaPipe 버전 호환 문제

- 문제 : mediapipe 0.10.30 이후 mp.solutions(Hands) 제거로 실행 불가
- 해결 : mediapipe==0.10.21 + protobuf<5, numpy<2 함께 고정

Streamlit 배포 실패

- 문제 : 클라우드 기본 Python 3.14 → mediapipe wheel 없음
- 해결 : 앱 삭제 후 Python 3.12 선택해 재배포



배포 실패 화면 (Python 3.14)

6. 보완사항 및 개선점

데이터 규모 및 다양성 확대

- 240개(4세션) 한 사람 데이터 → 여러 사람·조명·각도로 일반화 검증
- 오답이 많은 숫자(1·7, 4·9 등) 중심으로 추가 수집

조기 예측 성능 개선

- 50% 지점 정확도 0.44~0.56 → 앞부분 증강·하이퍼파라미터 탐색
- 자동 입력과 연결 + 틀린 예측을 바로 고치는 UI 추가

학습 · 실행 속도 최적화

- CPU 학습으로 오래 걸림 → GPU(Colab) 학습, --quick으로 먼저 검증
- ONNX 변환·양자화로 앱 지연 단축, 여러 자리 수·소수점 지원

7. 소감 및 후기

프로젝트 성과

웹캠 하나로 데이터 수집부터 전처리, CNN+BiGRU 하이브리드 학습, 조기 예측 모델 비교, 웹앱 배포까지 AI 서비스 개발의 전 과정을 직접 경험했습니다.

교차검증 macro F1 0.961, 처음 보는 확인용 세션에서도 0.917을 얻었고, MNIST 사전학습과 세션 단위 평가처럼 적은 데이터를 다루는 방법을 배웠습니다.

아쉬웠던 점

하이브리드 모델이다 보니 모델이 무거워 데이터 수집과 학습 과정이 전반적으로 오래 걸렸습니다.

저녁 8시에 시작한 작업이 새벽 3시에야 끝났고, 조기 예측 실험만 약 28분(1,662초)이 걸려 여러 설정을 충분히 시도하지 못했습니다. 다음에는 GPU 환경과 작은 실험으로 먼저 검증하는 방식으로 시간을 줄이고 싶습니다.